Sprint 2 Planning Meeting: Smart Expense Tracker

1. Executive Meeting Overview

The Sprint 2 Planning Meeting marks a critical evolution in the Smart Expense Tracker project lifecycle. Having successfully met all foundational deliverables in Sprint 1, the team is now moving from simple feature implementation toward a deeper phase of strategic problem-solving. Under the Lead's direction, the focus has shifted from "setting deliverables" to establishing a robust architectural framework that addresses the core technical challenges of financial data reconciliation. This sprint is centered on the "crux" of the application: the transformation of opaque, raw bank data into high-utility, human-readable objects through a sophisticated tagging and manual review architecture. The following log details the collaborative brainstorming and technical decision-making that will define our execution over the next four weeks.

2. Formatted Meeting Dialogue

Lead (Supervisor): We have successfully cleared Sprint 1, meeting every deliverable. For Sprint 2, I am not handing you a pre-written requirement doc. Instead, we are going to generate these deliverables together. We need to transition from just "building things" to strategic planning. We need to decide what is required to reach the final vision of a smart finance tracker.
Technical Team Member: I believe we should start with remediation. I had a design for the UI regarding uploads and scans that needs refinement to be truly production-ready.
Technical Team Member: We also need to address the registration flow. Currently, bank accounts are automatically generated in the backend, but we need a legitimate onboarding process where users create an account and populate the database with their specific details.
Lead: Exactly. The registration flow must prompt for initial bank account details. The app is non-functional without an account. We should also use the "last four digits of the card" as a standard verification method to identify the source without storing sensitive PII.
Technical Team Member: Architecturally, should we keep the upload interface and the records table on a single page, or move toward a centralized history?
Technical Team Member: A centralized document management page is far superior. It allows users to view, delete, and manage all past uploads—receipts and statements alike—in one location. This improves the UI and allows us to verify parsing accuracy by comparing different documents side-by-side.
Lead: Let's discuss the real core of this application. Bank statements consist of "ugly" strings—they are not human-friendly. This app is essentially a "bank-to-human" translator. We are taking raw strings and converting them into powerful metadata objects with tags, descriptions, and verified fields.

Technical Team Member: That brings us to the tagging system. We need to research how to bridge those many-to-many relationships from the backend to the frontend.
Lead: The tagging system is the crux. It must function like Instagram-style hashtags. Users need the ability to define their own tags so they can ask granular questions, such as "How much did I spend on groceries?" across multiple vendors.
Technical Team Member: So, the user will be responsible for manually tagging these transactions after they are parsed into the system?
Lead: Initially, yes. In this phase, everything must pass through a Manual Review Queue. This handles the "Steam purchase" problem where raw rows are non-descriptive. The journey is: Upload, Parse, Review, and Tag. Later, we can look at pattern recognition to automate this, but we must build the manual structure first.The dialogue above establishes a roadmap where data integrity and user-controlled metadata take precedence, directly informing the deliverables outlined in the next section.

3. Sprint 2 Deliverables: Core Objectives

These deliverables are the strategic pillars required to transform raw financial data into actionable insights. By focusing on the relationship between documents and custom metadata, we ensure the "Smart Finance Tracker" can support complex user queries and future automation.

Deliverable Name	Description & Impact	Technical Requirements
Instagram-Style Tagging System	The application's "Crux." Enables granular, user-defined labels for expenses. Impact: Allows personalized financial analysis (e.g., #groceries vs #woolies).	Implementation of Junction Tables to support Many-to-Many relationships; unique string-based Tag IDs per user.
Centralized Document Management	A unified "History" page for managing all source files. Impact: Centralizes control, allowing users to audit, view, and delete Receipts vs. Statements in one UI.	Backend integration of the "Unified Response Envelope" for consistent JSON output; React state lifting for bulk data handling.
User Onboarding & Account Logic	A formal registration flow. Impact: Establishes the required data hierarchy (User -> Account -> Transaction) before the app is accessible.	Logic to populate the account_number_mask using the last four digits of the card; validation for "initial bank account" setup.
Bank-to-Human Transformation Engine	The logic layer that converts raw CSV/PDF strings into rich objects. Impact: Essential for making "ugly"	Schema mapping for Canonical Payload fields: category, merchant_name,

Deliverable Name	Description & Impact	Technical Requirements
	bank rows understandable and searchable for the user.	amount (using Decimal types), and transaction_date (ISO 8601).

These core goals prioritize structural stability, ensuring that the underlying database can handle high-precision financial data before we pursue higher-level performance targets.

4. Sprint 2 Stretch Goals: Enhanced Capabilities

Stretch goals serve as a bridge to Sprint 3, focusing on scalability and the transition from manual workflows to intelligent assistance.

- **Mobile Web Optimization:** Assessing browser performance on mobile devices. The goal is to ensure the React interface is responsive and functional on mobile browsers, negating the immediate need for a native APK.
- **Deployment Exploration:** Moving the application beyond a local Docker environment. This involves researching cloud hosting or server-based environments to support remote access.
- **Automated Pattern Recognition:** Initial research into "pre-filling" fields. This involves identifying recurring raw strings (e.g., "Woolworths Stellenbosch") and suggesting locations or tags based on historical user data.These goals remain contingent on the successful stabilization of the core relational database and the manual review architecture.

5. Suggested Implementation Roadmap

The sequencing of these tasks follows a "back-to-front" development philosophy, ensuring the data architecture is verified before final UI components are locked in.

Phase I: Remediation & Polish
Address "polishing work" identified in Sprint 1. This involves cleaning up the existing UI for uploads and ensuring the basic ingestion flow is seamless. We will finalize the "Upload Box" design to be more intuitive for both file types.

Phase II: Data Architecture (The "Crux")
Establish the relational database schema for Many-to-Many tagging. This requires creating the Tags table and the Transaction_Tags junction table. This structure is vital for allowing a single transaction to hold multiple user-defined hashtags.

Phase III: Onboarding & Account Logic

Implement the user registration flow. This phase focuses on the "Account" parent table, specifically capturing the institution name and the account_number_mask (last four digits) to support source verification while maintaining PII security.

Phase IV: Centralized Document Management
Construct the "History" page UI. By utilizing the "Unified Response Envelope" from the backend, the frontend will be able to display a consistent list of all uploaded documents regardless of whether they are receipts or bank statements, including global delete and view capabilities.

Phase V: Human-Readable Transformation
Develop the logic to map raw strings to the Canonical Multipart Upload Payload. This engine will extract merchant_name, transaction_date (ISO 8601), and amount. To maintain data integrity, the amount field will strictly use Decimal types (DECIMAL 12, 2) rather than floats to prevent rounding errors.This sequence ensures the underlying structure is finalized before higher-level reporting and dashboards are attempted in the final sprint.

6. Summary of Architectural Decisions

The meeting concluded with three fundamental architectural decisions that will guide all technical development in Sprint 2:

1. **The "Many-to-Many" Tagging Approach:** The team rejected simple comma-separated string columns in favor of a robust junction table. This decision supports the "Instagram-style" requirement, allowing for complex multi-tag filtering and preventing data normalization issues.
2. **The "Manual Review Queue" Necessity:** To solve the problem of non-descriptive bank rows (the "Steam purchase" issue), all parsed data will initially land in a review queue. This forces a "Review and Tag" step, ensuring that metadata is verified by the user before being finalized for reporting.
3. **The "Bank-to-Human" Metadata Format:** The system's value proposition is defined as its ability to translate raw strings into metadata-rich objects. All backend logic must prioritize the creation of these objects, ensuring they contain standardized fields (Category, Merchant, Amount, Date) to serve as the primary units of analysis for future dashboards.